

Exact Subset Sum: Reproducible Baselines and Measurements

Le Duc Minh
minh.leduc.0210@gmail.com

Abstract—Subset sum asks whether some of the given integers add up to a target value. The best known algorithm that works on every input takes about $2^{n/2}$ steps for n integers, and nobody knows whether $2^{n/3}$ is possible. This note does not answer that question. It sets up a clean, checked starting point for anyone who wants to work on it. We explain the problem and the notation, then describe and prove correct three exact solvers: meet-in-the-middle, a dynamic program over all reachable sums, and the Schroeppe–Shamir algorithm. On inputs with no solution, the measured running time grows like $2^{n/2}$ for both meet-in-the-middle and Schroeppe–Shamir, and Schroeppe–Shamir uses memory that grows only like $2^{n/4}$. We also note the easy case where the numbers are small: if their absolute values add up to at most $2^{n/3}$, the dynamic program already runs in about $2^{n/3}$ steps. That case is textbook material and says nothing about the general question.

Index Terms—subset sum, knapsack, exact algorithms, meet-in-the-middle, Schroeppe–Shamir

I. INTRODUCTION

Suppose you have n integers and a target. Can you pick some of them so that the picked numbers add up exactly to the target? This is the *subset sum* problem. For the numbers 3, 5, 8, 13 and target 16 the answer is yes, in two ways: $3 + 13$ and $3 + 5 + 8$. For target 2 the answer is no.

Written as an equation, given integers $a_1, \dots, a_n$ and b , we ask whether

$$\exists x \in \{0, 1\}^n : \sum_{i=1}^n a_i x_i = b. \quad (1)$$

Each x_i says whether a_i is picked ($x_i = 1$) or not ($x_i = 0$). This is the same as the yes/no version of the 0–1 knapsack problem.

A. Why it matters

Subset sum is one of the standard NP-complete problems, so nobody expects a polynomial-time algorithm. It also shows up in practice. Early public-key schemes such as the Merkle–Hellman knapsack cryptosystem [8] based their security on it, and ideas from subset sum algorithms are still used in attacks on code-based and lattice-based schemes. Many budgeting and packing questions reduce to it as well.

Because a polynomial algorithm is not expected, the interesting question is how fast the exponential part can be. Trying every subset costs 2^n steps. The meet-in-the-middle method of Horowitz and Sahni [1] brings this down to about $2^{n/2}$, and that is still the best known bound that holds for every input, 50 years later. The difference between exponents is large in

practice. For $n = 60$, $2^{n/2}$ is about 10^9 while $2^{n/3}$ is about 10^6 . For $n = 100$ the gap is about 10^{15} versus 10^{10} .

B. What is known

Schroeppe and Shamir [2] kept the $2^{n/2}$ time and cut the memory to $2^{n/4}$. Chen, Jin, Randolph and Servedio [5] shaved a polynomial factor off the time, to $2^{n/2}/n^{0.5023}$, but the exponent is still $1/2$. On *random* inputs, the representation technique of Howgrave-Graham and Joux [3] and Becker, Coron and Joux [4] is much faster, about $2^{0.291n}$, but those bounds rely on assumptions about how random inputs behave and do not hold for every input. Randolph and Węgrzycki [7] recently beat $2^{n/2}$ for every input on several related problems, but not for subset sum itself. So whether $2^{n/3}$ (or anything below $2^{n/2}$) is possible for every input is still open. We make no claim about it.

C. What this note does

This note covers the problem, the notation, and the classical exact algorithms, with proofs and measurements. Its contents are:

- a self-contained introduction to the problem and its notation (Section II);
- correctness and running-time proofs for meet-in-the-middle, a signed dynamic program, and Schroeppe–Shamir (Sections III–VI);
- measurements of running time and memory on inputs with no solution (Section VII).

Section VIII states what a real $2^{n/3}$ result would have to show.

II. PRELIMINARIES

This section collects the definitions used in the rest of the paper. Readers who already know the area can skip it.

A. Inputs

The input is a list $a_1, \dots, a_n$ of integers and a target b . We allow negative numbers and a negative target. If the same value appears twice, the two copies count as different items, and each item can be used at most once. The empty selection is allowed and has sum 0, so the answer for $b = 0$ is always yes.

A solver can answer in two ways. The *decision* answer is just yes or no. The *search* answer also returns a *witness*: the positions of the picked items. A witness is easy to check by adding the numbers, so every solver in this note returns one when the answer is yes.

B. Subsets and their sums

We write $[n] = \{1, \dots, n\}$ for the positions. For a set of positions $S \subseteq [n]$ let $w(S) = \sum_{i \in S} a_i$ be its sum. There are 2^n subsets, so there are 2^n subset sums, although some of them may be equal.

For a list A we write $S(A)$ for the list of all its subset sums, keeping repeats. It is built one item at a time:

$$S([]) = [0], \quad (2)$$

$$S(a :: A) = S(A) ++ [a + s : s \in S(A)]. \quad (3)$$

Here $a :: A$ is the list with first item a and rest A , and $++$ joins two lists. Each step doubles the length, so $S(A)$ has $2^{|A|}$ entries.

C. Measuring cost

We count running time as a function of n , the number of items. The size of each number matters too. Adding two 1000-bit numbers costs more than adding two small ones. We write $\mathcal{O}^*(f(n))$ for a bound of the form $f(n) \cdot p$, where p is a polynomial in n and in the number of bits of the input. For example, $\mathcal{O}^*(2^{n/2})$ allows $n^2 \cdot 2^{n/2}$ steps, and also allows the cost of handling large numbers. We use this notation because the open question is about the exponent; polynomial factors do not change it.

D. Worst case, average case, and small numbers

A *worst-case* bound must hold for every input of size n . An *average-case* bound only holds for inputs drawn from some random distribution, usually with a small chance of failure. The $2^{n/3}$ question is about the worst case.

The size of the numbers also matters. Let $W = \sum_i |a_i|$. Every subset sum lies between $-W$ and W , so a table with $2W + 1$ cells is enough to record which sums are reachable. This gives an algorithm with about nW steps (Section IV). Such a bound is called *pseudopolynomial*: it is fast when the numbers are small and slow when they are large. The hard inputs are therefore the ones with large numbers, roughly n bits each. Inputs whose numbers are much larger are easy for a different reason: powers of two such as $1, 2, 4, 8, \dots$ can be solved by reading the binary digits of the target.

E. A worked example of meet-in-the-middle

The idea behind the $2^{n/2}$ bound fits in one example. Take $a = (3, 5, 8, 13)$ and $b = 16$. Split the items into a left half $(3, 5)$ and a right half $(8, 13)$. List all subset sums of each half:

$$\text{left: } 0, 3, 5, 8 \quad \text{right: } 0, 8, 13, 21.$$

A solution picks some items from each half, so it exists exactly when some left sum u and right sum v satisfy $u + v = 16$. Sort the right list. For each left sum u , look up $16 - u$ with binary search: $u = 3$ finds 13, and $u = 8$ finds 8. These are the two solutions $3 + 13$ and $3 + 5 + 8$. Each half has only $2^{n/2}$ sums, so the whole search costs about $2^{n/2}$ steps instead of 2^n . Section III turns this into a proof.

Algorithm 1 Sorted meet-in-the-middle

Require: numbers $a_1, \dots, a_n$; target b

Ensure: positions of a subset summing to b , or $\perp$ if none exists

```

1:  $k \leftarrow \lfloor n/2 \rfloor$ 
2:  $L \leftarrow \text{ENUMERATE}(a_1, \dots, a_k)$ 
3:  $R \leftarrow \text{SORTBYSUM}(\text{ENUMERATE}(a_{k+1}, \dots, a_n))$ 
4: for  $(u, m_L) \in L$  do
5:    $p \leftarrow$  first position in  $R$  with sum  $\geq b - u$   $\triangleright$  binary search
6:   if  $p \leq |R|$  and  $R[p].\text{sum} = b - u$  then
7:     return the indices in  $m_L \mid (R[p].\text{mask} \ll k)$ 
8:   end if
9: end for
10: return  $\perp$ 

```

III. ENUMERATION AND MEET-IN-THE-MIDDLE

We first show that the list $S(A)$ captures exactly the reachable sums, and that a solution splits cleanly across any division of the items. All proofs are in Appendix A.

Lemma 1 (Enumeration correctness). *$b \in S(A)$ if and only if A has a 0-1 selection summing to b . Also, $|S(A)| = 2^{|A|}$.*

Lemma 2 (Split correctness). *For any lists L, R , a selection of $L ++ R$ sums to b if and only if there are $u \in S(L)$ and $v \in S(R)$ with $u + v = b$.*

Algorithm 1 is the worked example of Section II in general form. Each sum carries a bit mask that records which items produced it, so a match gives a witness directly.

Theorem 1 (General baseline). *There is a deterministic algorithm that solves (1) on every input, returns a witness when one exists, and uses $\mathcal{O}^*(2^{n/2})$ time and memory.*

IV. SIGNED DYNAMIC PROGRAMMING

When the numbers are small, a table of reachable sums is faster than listing subsets. Let $W = \sum_i |a_i|$. Every partial sum lies in $[-W, W]$, so we keep one cell per value in that range. A cell is either empty or holds a bit mask of a subset that reaches that value. At the start only the cell for 0 is filled. Item a_i updates the table by

$$D_i(s) = D_{i-1}(s) \vee D_{i-1}(s - a_i), \quad (4)$$

where cells outside $[-W, W]$ count as empty. In words, a value is reachable after item i if it was reachable before, or if it is a_i more than a value that was reachable before. The new table is written to a separate buffer, so an item is never used twice. Algorithm 2 gives the details.

Theorem 2 (Signed dynamic programming). *Algorithm 2 solves (1) correctly and returns a valid witness after $\mathcal{O}(n(2W + 1))$ cell visits.*

Algorithm 2 Dense signed-sum dynamic programming

Require: numbers $a_1, \dots, a_n$; target b **Ensure:** positions of a subset summing to b , or $\perp$ if none exists

```

1:  $W \leftarrow \sum_i |a_i|$ 
2: if  $|b| > W$  then
3:   return  $\perp$ 
4: end if
5:  $D \leftarrow [\perp] \times (2W + 1)$ ;  $D[W] \leftarrow 0$ 
   cell  $p$  represents sum  $p - W$ 
6: for  $i \leftarrow 1$  to  $n$  do
7:    $D' \leftarrow \text{copy of } D$ 
8:   for position  $p$  with  $D[p] \neq \perp$  do
9:      $q \leftarrow p + a_i$ 
10:    if  $0 \leq q \leq 2W$  and  $D'[q] = \perp$  then
11:       $D'[q] \leftarrow D[p] \mid (1 \ll (i - 1))$ 
12:    end if
13:  end for
14:   $D \leftarrow D'$ 
15: end for
16: return the indices in  $D[b + W]$ , or  $\perp$  if unreachable

```

V. THE EASY ONE-THIRD CASE

The dynamic program gives a $2^{n/3}$ bound when the numbers are small. We state it to make clear what it does and does not cover.

Theorem 3 (Small numbers). *If $W \leq 2^{\lfloor n/3 \rfloor}$, then (1) can be solved in $\mathcal{O}^*(2^{n/3})$ time.*

This is the standard pseudopolynomial argument and not a new algorithm. It does not reach the hard inputs. For $a_i = 2^{i-1}$, for example, $W = 2^n - 1$, far above the assumption. Those powers of two also give 2^n different subset sums, so removing duplicate sums cannot help in general. Splitting the items into three blocks of $n/3$ does not help either: each block is cheap to list, but combining two block lists naively already produces $2^{2n/3}$ pairs. These remarks rule out particular approaches. They are not lower bounds.

VI. SCHROEPPEL–SHAMIR

Meet-in-the-middle stores $2^{n/2}$ sums. Schroeppeel and Shamir get the same time with far less memory. They split the items into four blocks and generate the sums of block pairs one at a time instead of storing them. Each of the four block lists has about $2^{n/4}$ entries and is sorted. A heap produces the sums of blocks 0 and 1 in increasing order, and a second heap produces the sums of blocks 2 and 3 in decreasing order. Two pointers then walk the two streams toward each other, as in the classic search for two numbers with a given sum in sorted arrays (Algorithm 3). Each stream moves at most $2^{n/2}$ times.

Theorem 4 (Schroeppeel–Shamir). *Algorithm 3 correctly decides (1) and returns a valid witness in $\mathcal{O}^*(2^{n/2})$ time and $\mathcal{O}^*(2^{n/4})$ space.*

Algorithm 3 Schroeppeel–Shamir with streamed pair sums

Require: numbers $a_1, \dots, a_n$; target b **Ensure:** positions of a subset summing to b , or $\perp$ if none exists

```

1: partition the entries into four contiguous blocks
    $G_0, \dots, G_3$  of sizes  $\lfloor n/4 \rfloor$  or  $\lceil n/4 \rceil$ 
2: for  $t \leftarrow 0$  to  $3$  do
3:    $S_t \leftarrow \text{SORTBYSUM}(\text{ENUMERATE}(G_t))$   $\triangleright$  masks use
   global positions
4: end for
5:  $U \leftarrow \text{MERGEASCENDING}(S_0, S_1)$   $\triangleright$  streams  $u_0 + u_1$ 
   increasing
6:  $V \leftarrow \text{MERGEDESCENDING}(S_2, S_3)$   $\triangleright$  streams  $v_0 + v_1$ 
   decreasing
7:  $u \leftarrow \text{next}(U)$ ;  $v \leftarrow \text{next}(V)$ 
8: while  $u \neq \perp$  and  $v \neq \perp$  do
9:   if  $u.\text{sum} + v.\text{sum} = b$  then
10:    return the indices in  $u.\text{mask} \mid v.\text{mask}$ 
11:   else if  $u.\text{sum} + v.\text{sum} < b$  then
12:      $u \leftarrow \text{next}(U)$ 
13:   else
14:      $v \leftarrow \text{next}(V)$ 
15:   end if
16: end while
17: return  $\perp$ 

```

VII. MEASUREMENTS

We run the three methods on inputs that have no solution, because a solver may stop early when it finds a match and that would hide its real cost. To build such inputs, every number is even and the target is odd, so no subset can hit it. We use two families: *bounded* numbers in $[-100, 100]$, where the dynamic program’s table fits in memory, and *wide* numbers in $[-2^{40}, 2^{40}]$, where it does not. Each time is the best of three runs, and memory is the peak allocated during the run. All runs are single-threaded, on CPython 3.12 under Linux with an Intel Core i9-8950HK CPU at 2.9 GHz. Figure 2 also reports a machine-independent *work count*: the number of half-list sums built for meet-in-the-middle, the number of heap steps for Schroeppeel–Shamir, and the number of table cells visited for the dynamic program.

Tables I and II and Figures 1–3 report the results.

Meet-in-the-middle and Schroeppeel–Shamir take about the same time, and both grow like $2^{n/2}$. Schroeppeel–Shamir is a little slower because of its heap operations. Their memory use is very different: meet-in-the-middle grows like $2^{n/2}$, while Schroeppeel–Shamir grows like $2^{n/4}$ and uses about 200 times less memory at $n = 32$ (Table II). Nederlof and Węgrzycki [6] show that the memory can be pushed slightly below $2^{n/4}$ with randomization, at the same $2^{n/2}$ time. On bounded numbers the dynamic program’s time grows roughly linearly in n , and from $n = 28$ on it is the fastest method. On wide numbers it cannot run at all, because the table would need on the order of $n \cdot 2^{40}$ cells. Its peak memory is not monotone in n because

TABLE I
RUNTIME IN MILLISECONDS (BEST OF THE TIMED RUNS) ON INPUTS WITH NO SOLUTION; “—” MARKS A CONFIGURATION NOT RUN. BOUNDED VALUES ARE EVEN INTEGERS IN $[-100, 100]$, WIDE VALUES IN $[-2^{40}, 2^{40}]$; THE ODD TARGET FORCES EACH SOLVER TO RUN TO COMPLETION.

n	bounded values			wide values		
	DP	MITM	Schroeppel–Shamir	MITM	Schroeppel–Shamir	
8	0.87	0.01	0.02	0.01	0.03	
12	2.20	0.04	0.09	0.05	0.11	
16	4.37	0.21	0.38	0.19	0.38	
20	9.88	0.79	1.58	0.99	1.69	
24	11.10	4.31	6.40	4.65	6.92	
28	10.18	31.87	39.57	30.37	31.07	
32	18.35	123.53	151.87	140.81	150.72	
36	22.53	602.75	636.24	689.86	670.18	

TABLE II
PEAK TRACED MEMORY IN KiB ON BOUNDED-VALUE INSTANCES (LOWER IS BETTER). MITM GROWS LIKE $2^{n/2}$; SCHROEPEL–SHAMIR LIKE $2^{n/4}$.

n	DP	MITM	Schroeppel–Shamir
8	28.10	1.00	1.20
12	51.60	3.30	2.20
16	77.90	17.20	4.40
20	101.90	119.70	9.40
24	130.80	891.90	19.40
28	96.20	3855.40	35.90
32	136.40	15352.90	71.20

the table width $2W + 1$ depends on the random input. These are finite measurements. They match the proven bounds but do not prove anything about larger n .

VIII. WHAT A $2^{n/3}$ RESULT WOULD NEED

A real improvement needs an algorithm that provably finishes in $\mathcal{O}^*(2^{n/3})$ steps on *every* input, with the cost of arithmetic on large numbers counted. If it uses randomness, the proof must also bound its chance of failure. Good benchmark numbers cannot replace that proof, and neither can an average-case analysis, because both leave out the hardest inputs. The closest recent progress is the worst-case result of Randolph and Węgrzycki [7] for related problems, which uses the representation technique; whether it can be adapted to plain subset sum is open. The algorithms in this note are the baseline that any such result has to beat.

APPENDIX A PROOFS

This appendix proves the results stated in the main text. Throughout, a *selection* of a list is a choice of some of its positions, each used at most once, and its sum is the sum of the chosen entries.

A. Proof of Lemma 1

We use induction on the length of A .

Base case. If A is empty, its only selection is the empty one, with sum 0, and $S([]) = [0]$. So $b \in S([])$ exactly when $b = 0$, and $|S([])| = 1 = 2^0$.

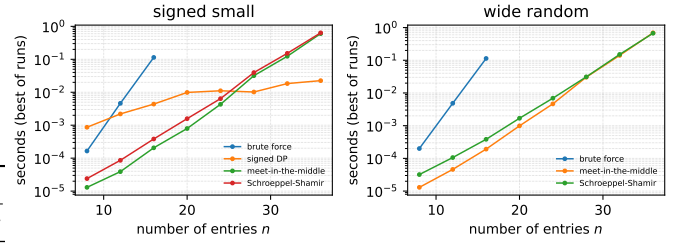


Fig. 1. Measured runtime versus n on inputs with no solution, log scale. Wide values (right) admit no pseudopolynomial table, and meet-in-the-middle and Schroeppel–Shamir both grow exponentially; bounded values (left) are solved by the signed DP in time that grows only slowly with n .

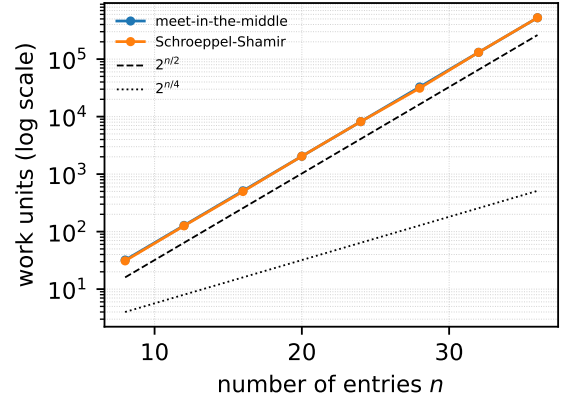


Fig. 2. Work count (defined in Section VII) versus n (log scale) with the reference slopes $2^{n/2}$ and $2^{n/4}$. The measured counts track $2^{n/2}$ as predicted; the $2^{n/4}$ line bounds the space, not the time.

Step. Let $A = a :: A'$ and assume the claim for A' . Every selection of A either leaves out the first entry or uses it.

- If it leaves it out, it is a selection of A' . By the induction hypothesis its sum b is in $S(A')$, which is the first part of $S(A)$.
- If it uses it, the rest is a selection of A' with some sum s , and $b = a + s$. Since $s \in S(A')$, the value $a + s$ is in the second part of $S(A)$.

Conversely, a value in the first part of $S(A)$ is a sum of some selection of A' , which is also a selection of A . A value $a + s$ in the second part comes from a selection of A' with sum s ; adding the first entry gives a selection of A with sum $a + s$. The first entry was not used before, so no position is used twice. This proves the equivalence.

For the length, $|S(A)| = |S(A')| + |S(A')| = 2 \cdot 2^{|A'|} = 2^{|A|}$. The list keeps repeated values, so equal sums do not change the count. $\square$

B. Proof of Lemma 2

The positions of $L++R$ are the positions of L followed by the positions of R , and the two groups do not overlap.

Suppose a selection of $L++R$ has sum b . Let u be the sum of its chosen positions inside L and v the sum of its chosen positions inside R . Then $u + v = b$. The positions inside L

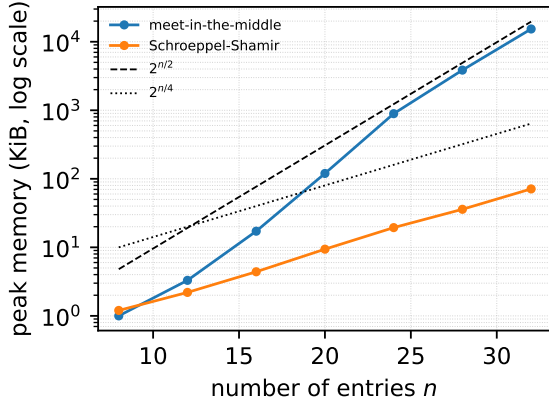


Fig. 3. Peak memory versus n (log scale). Meet-in-the-middle grows by about a factor of four per four added entries ($2^{n/2}$); Schroeppel-Shamir grows by about a factor of two ($2^{n/4}$), as its analysis predicts.

form a selection of L , so $u \in S(L)$ by Lemma 1; in the same way $v \in S(R)$.

Now suppose $u \in S(L)$ and $v \in S(R)$ with $u + v = b$. By Lemma 1 there is a selection of L with sum u and a selection of R with sum v . Taking both together gives a selection of $L + R$ with sum $u + v = b$. No position is used twice because the two groups are disjoint. The argument does not use the signs or the distinctness of the entries. $\square$

C. Proof of Theorem 1

We show that Algorithm 1 has the stated properties.

Correctness. Let $k = \lfloor n/2 \rfloor$, $L = (a_1, \dots, a_k)$ and $R = (a_{k+1}, \dots, a_n)$. By Lemma 2, a solution exists if and only if some $u \in S(L)$ and $v \in S(R)$ satisfy $v = b - u$. The algorithm tries every $u \in S(L)$ and uses binary search on the sorted list $S(R)$ to decide whether $b - u$ is present. Binary search on a sorted list finds a value exactly when it is present, so the algorithm answers yes exactly when a solution exists.

Witness. Each entry of the two lists stores, next to its sum, a bit mask of the positions that produced it. When u and $v = b - u$ match, the left mask describes a selection of L with sum u and the right mask, shifted by k , describes a selection of R with sum v . Their union is a selection of the whole input with sum b .

Cost. Let $P = 2^k$ and $Q = 2^{n-k}$ be the list lengths (Lemma 1). Building the lists takes $\mathcal{O}(P + Q)$ additions and mask updates. Sorting the right list takes $\mathcal{O}(Q \log Q)$ comparisons, and the P binary searches take $\mathcal{O}(P \log(Q + 1))$ comparisons. Since k and $n - k$ are at most $\lceil n/2 \rceil$, both P and Q are at most $2^{\lceil n/2 \rceil}$ and $\log Q \leq n$, so the total is $\mathcal{O}(n 2^{\lceil n/2 \rceil})$ operations on $\mathcal{O}(2^{\lceil n/2 \rceil})$ stored records. Each sum has absolute value at most $\sum_i |a_i|$, so its bit length is at most the input length plus $\log_2 n$, and each mask has n bits. Every operation therefore costs a polynomial factor, which gives $\mathcal{O}^*(2^{n/2})$ time and memory. The algorithm makes no random choices. $\square$

D. Proof of Theorem 2

Write D_i for the table after the first i items. We prove the following invariant by induction on i :

($\star$) For every $s \in [-W, W]$, the cell for s in D_i is filled if and only if some selection of $a_1, \dots, a_i$ has sum s , and in that case the stored mask is such a selection.

Base case. With no items, the only selection is the empty one, with sum 0. The algorithm fills only the cell for 0, with the empty mask.

Step. Assume ($\star$) for D_{i-1} . A selection of $a_1, \dots, a_i$ either skips a_i or uses it. If it skips a_i , it is a selection of the first $i - 1$ items, so its sum s is filled in D_{i-1} and the copy keeps it in D_i . If it uses a_i , the rest has sum $s - a_i$, which is filled in D_{i-1} ; the update writes the cell for s , with the old mask plus bit i . In both cases the cell for s is filled in D_i . Conversely, every cell filled in D_i was either copied from D_{i-1} (a selection that skips a_i) or written by the update from a filled cell $s - a_i$ (a selection that uses a_i , added once because the update reads only the old table). In both cases the stored mask is a selection with the right sum. This proves ($\star$) for D_i .

Every partial sum s of a selection satisfies $|s| \leq \sum_i |a_i| = W$, so no reachable value falls outside the table. By ($\star$) for D_n , the answer is yes exactly when the cell for b is filled, and its mask is a witness. If $|b| > W$ no selection can reach b , and the algorithm correctly answers no.

Cost. Each of the n rounds copies the table and scans its $2W + 1$ cells once, so the total is $\mathcal{O}(n(2W + 1))$ cell visits. $\square$

E. Proof of Theorem 3

Run Algorithm 2. It is correct by Theorem 2 and makes $\mathcal{O}(n(2W + 1))$ cell visits. If $W \leq 2^{\lfloor n/3 \rfloor}$, then

$$n(2W + 1) \leq n(2 \cdot 2^{\lfloor n/3 \rfloor} + 1) \leq 3n \cdot 2^{n/3}.$$

Each cell visit handles a mask of n bits and a sum of $\mathcal{O}(n)$ bits, so it costs a polynomial number of bit operations. The total is therefore $\mathcal{O}^*(2^{n/3})$. $\square$

F. Proof of Theorem 4

Reduction to two streams. Apply Lemma 2 three times: to split the input into $G_0 + G_1$ and $G_2 + G_3$, and then to split each of these. A solution exists if and only if there are $u_0 \in S(G_0)$, $u_1 \in S(G_1)$, $v_0 \in S(G_2)$ and $v_1 \in S(G_3)$ with $(u_0 + u_1) + (v_0 + v_1) = b$. Let U be the list of all values $u = u_0 + u_1$ and V the list of all values $v = v_0 + v_1$.

The streams are sorted. The heap for U holds one candidate for each $u_0 \in S(G_0)$: the next unused value $u_0 + u_1$, taken in increasing order of u_1 in the sorted list $S(G_1)$. Each such row is increasing, so the smallest heap entry is always the smallest value not yet output. Hence U is produced in increasing order, and it contains every pair exactly once. The same argument with the order reversed shows that V is produced in decreasing order.

The two-pointer walk is complete. Suppose $u^* \in U$ and $v^* \in V$ satisfy $u^* + v^* = b$, and suppose the walk stops

without a match. Then at some step it moves one pointer past its target value; consider the first such step.

- If the U pointer leaves u^* , the walk saw $u^* + v < b$ for the current v . Then $v < v^*$, so the V pointer had already moved past v^* , which contradicts the choice of the first step.
- If the V pointer leaves v^* , the walk saw $u + v^* > b$ for the current u . Then $u > u^*$, so the U pointer had already moved past u^* , again a contradiction.

So the walk stops with some pair whose sum is b . If no solution exists, no pair sums to b , and the walk ends when a stream runs out. The masks stored with u and v give a witness by the same argument as in Theorem 1.

Cost. The four sorted lists have $\mathcal{O}(2^{n/4})$ entries each, and each heap holds one entry per row, so memory is $\mathcal{O}(2^{n/4})$ records. Each stream has at most $2^{\lceil n/2 \rceil}$ values, so the walk takes at most $2 \cdot 2^{\lceil n/2 \rceil}$ steps. Each step is one heap operation costing $\mathcal{O}(\log 2^{n/4}) = \mathcal{O}(n)$ comparisons. With polynomial bit costs per operation, the time is $\mathcal{O}^*(2^{n/2})$ and the memory is $\mathcal{O}^*(2^{n/4})$. $\square$

REFERENCES

- [1] E. Horowitz and S. Sahni, “Computing partitions with applications to the knapsack problem,” *Journal of the ACM*, vol. 21, no. 2, pp. 277–292, 1974.
- [2] R. Schroeppe and A. Shamir, “A $T = \mathcal{O}(2^{n/2})$, $S = \mathcal{O}(2^{n/4})$ algorithm for certain NP-complete problems,” *SIAM Journal on Computing*, vol. 10, no. 3, pp. 456–464, 1981.
- [3] N. Howgrave-Graham and A. Joux, “New generic algorithms for hard knapsacks,” in *EUROCRYPT*, 2010.
- [4] A. Becker, J.-S. Coron, and A. Joux, “Improved generic algorithms for hard knapsacks,” in *EUROCRYPT*, 2011; IACR ePrint 2011/474.
- [5] X. Chen, Y. Jin, T. Randolph, and R. A. Servedio, “Subset sum in time $2^{n/2}/\text{poly}(n)$,” in *RANDOM*, 2023; arXiv:2301.07134.
- [6] J. Nederlof and K. Węgrzycki, “Improving Schroeppe and Shamir’s algorithm for subset sum via orthogonal vectors,” in *STOC*, 2021; arXiv:2010.08576.
- [7] T. Randolph and K. Węgrzycki, “Beating meet-in-the-middle for subset balancing problems,” arXiv:2511.10823, 2025.
- [8] R. Merkle and M. Hellman, “Hiding information and signatures in trapdoor knapsacks,” *IEEE Transactions on Information Theory*, vol. 24, no. 5, pp. 525–530, 1978.